

王道考研——组成原理

WWW.CSKAOYAN.COM

第四章 指令系统

考点13

指令格式

考点13：指令格式

	• 综合题43			• 综合题44	• 综合题44	• 综合题44		• 小题16
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题15	• 综合题45		• 综合题43	• 小题19	• 综合题44	• 大题43 • 大题44	• 小题16	
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频： 小题×4、综合题×9

指令格式

【考点笔记】指令的格式

为了区分各种数据寻址方式，在指令字中设一个字段，用来指明属于哪种寻址方式。

操作码 OP	寻址特征	形式地址 A
--------	------	--------

操作码的位数决定了指令的条数，寻址特征和形式地址 A 共同决定了可寻址的范围。

- 若为立即寻址，则形式地址 A 的位数决定了数的范围。
- 若为直接寻址，则形式地址 A 的位数决定了可寻址的范围。
- 若为寄存器寻址，则形式地址 A 的位数决定了通用寄存器的最大数量。
- 若为寄存器间接寻址，则寄存器字长决定了可寻址的范围。

考点14

指令的寻址方式

考点14：指令的寻址方式

• 小题16	• 综合题43	• 小题16 • 小题17		• 小题17 • 综合题44	• 小题17 • 综合题44		• 小题17	• 小题15
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题18	• 综合题45	• 小题16	• 综合题43		• 小题17 • 综合题44			
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频： 小题×10、综合题×6

指令的寻址方式

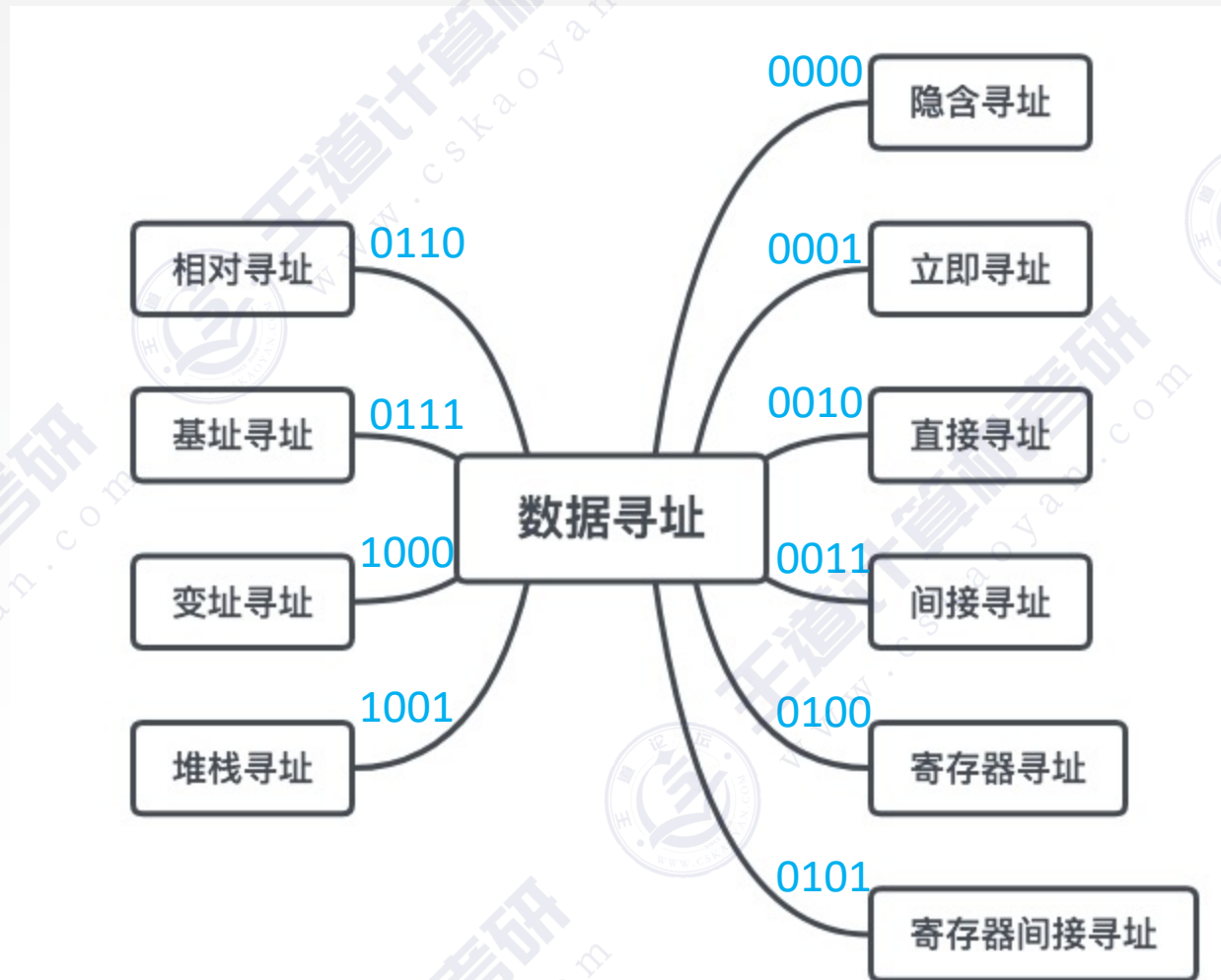
一地址指令

操作码 (OP)

寻址特征

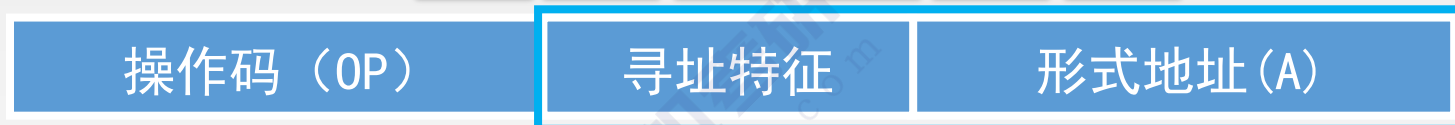
形式地址 (A)

求出操作数的真实地址，称为有效地址(EA)。



数据寻址

一地址指令



求出操作数的真实地址，称为有效地址(EA)。

二地址指令



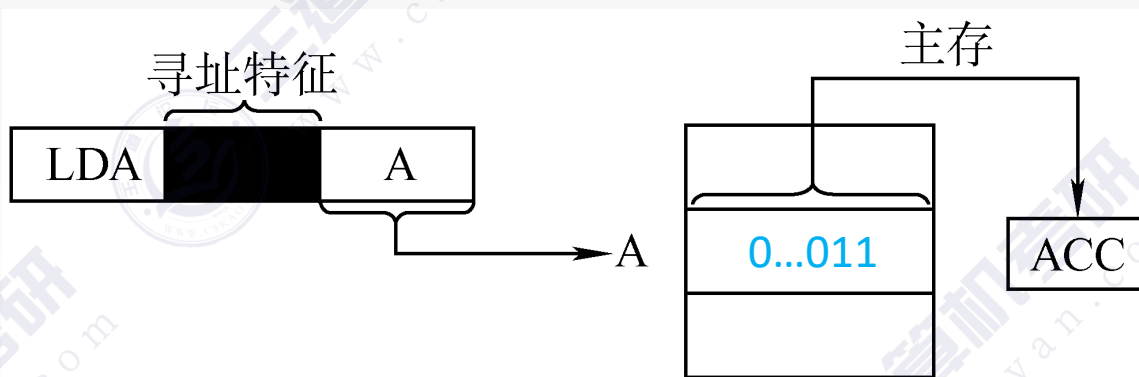
直接寻址

一地址指令

操作码 (OP)

1001...0111

直接寻址：指令字中的形式地址A就是操作数的真实地址EA，即 $EA=A$ 。



一条指令的执行：
取指令 访存1次
执行指令 访存1次
暂不考虑存结果
共访存2次

优点：简单，指令执行阶段仅访问一次主存，
不需专门计算操作数的地址。

缺点：
A的位数决定了该指令操作数的寻址范围。
操作数的地址不易修改。

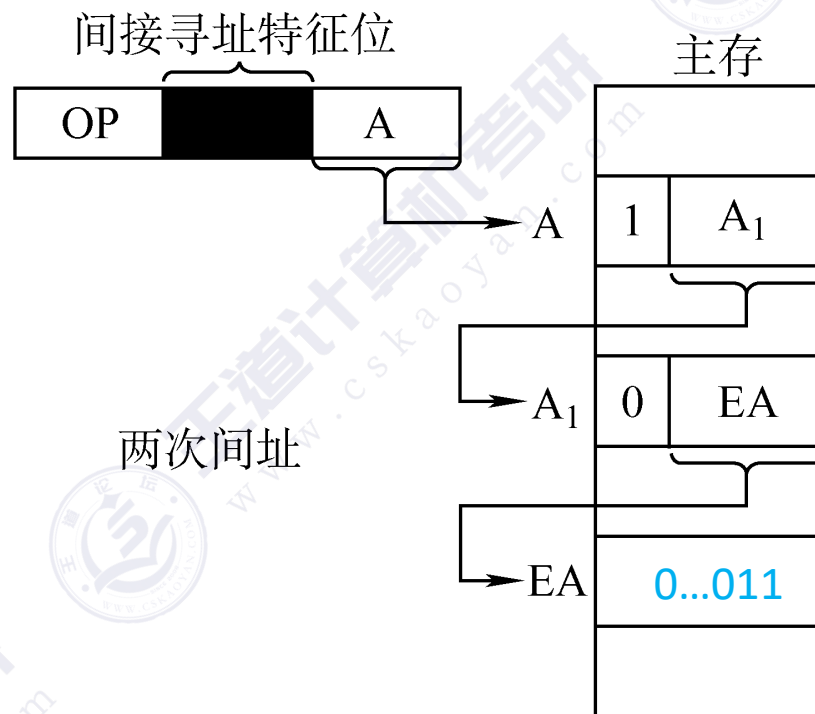
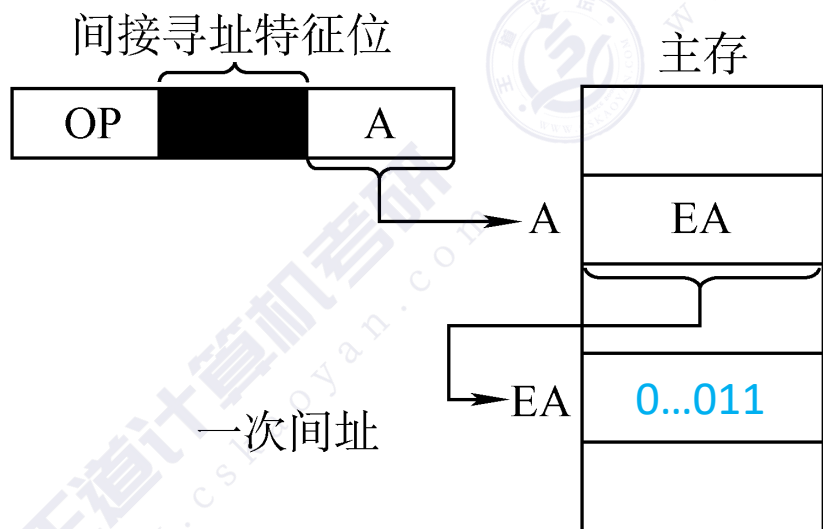
间接寻址

一地址指令

操作码 (OP)

1001...0111

间接寻址：指令的地址字段给出的形式地址不是操作数的真正地址，而是操作数有效地址所在的存储单元的地址，也就是操作数地址的地址，即 $EA=(A)$ 。



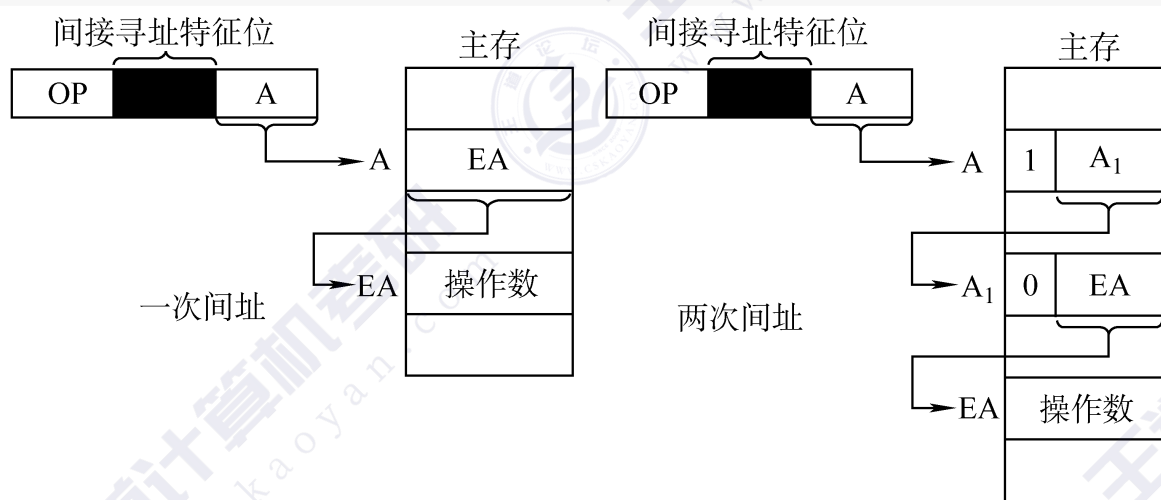
间接寻址

一地址指令

操作码 (OP)

1001...0111

间接寻址：指令的地址字段给出的形式地址不是操作数的真正地址，而是操作数有效地址所在的存储单元的地址，也就是操作数地址的地址，即 $EA=(A)$ 。



优点：

可扩大寻址范围(有效地址EA的位数大于形式地址A的位数)。

便于编制程序(用间接寻址可以方便地完成子程序返回)。

缺点：

指令在执行阶段要多次访存(一次间址需两次访存，多次寻址需根据存储字的最高位确定几次访存)。

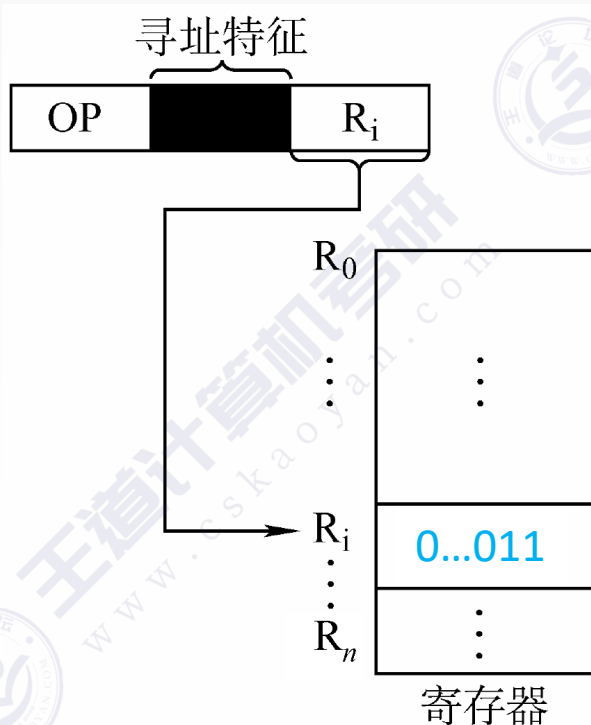
寄存器寻址

一地址指令

操作码 (OP)

1001

寄存器寻址：在指令字中直接给出操作数所在的寄存器编号，即 $EA = R_i$ ，其操作数在由 R_i 所指的寄存器内。



一条指令的执行：
取指令 访存1次
执行指令 访存0次
暂不考虑存结果
共访存1次

优点：

指令在执行阶段不访问主存，只访问寄存器，指令字短且执行速度快，支持向量/矩阵运算。

缺点：

寄存器价格昂贵，计算机中寄存器个数有限。

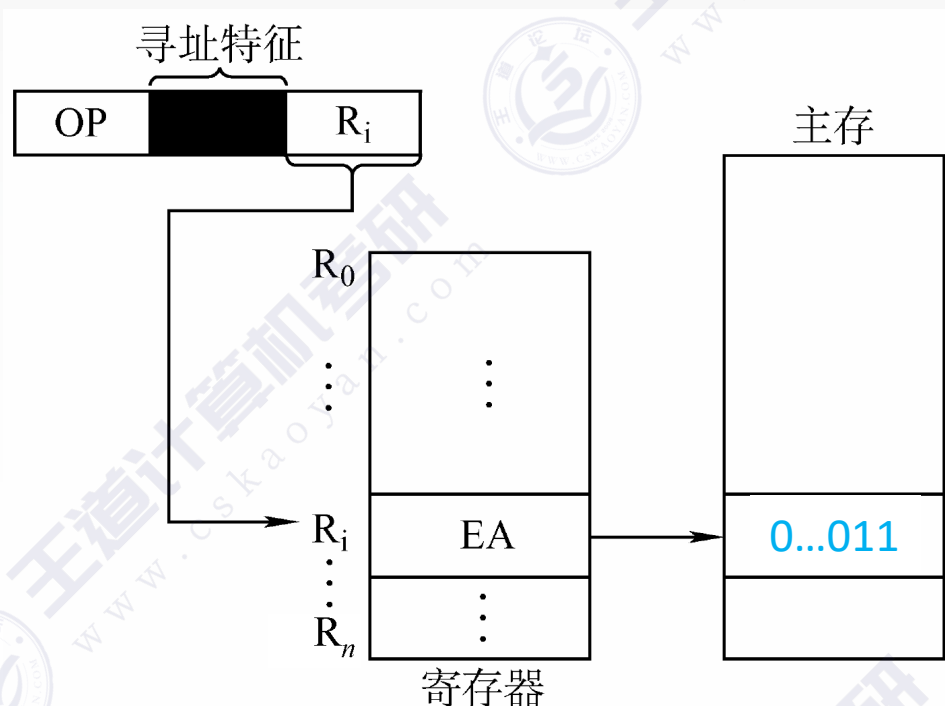
寄存器间接寻址

一地址指令

操作码 (OP)

1001

寄存器间接寻址：寄存器 R_i 中给出的不是一个操作数，而是操作数所在主存单元的地址，即 $EA=(R_i)$ 。



一条指令的执行：

取指令 访存1次

执行指令 访存1次

暂不考虑存结果

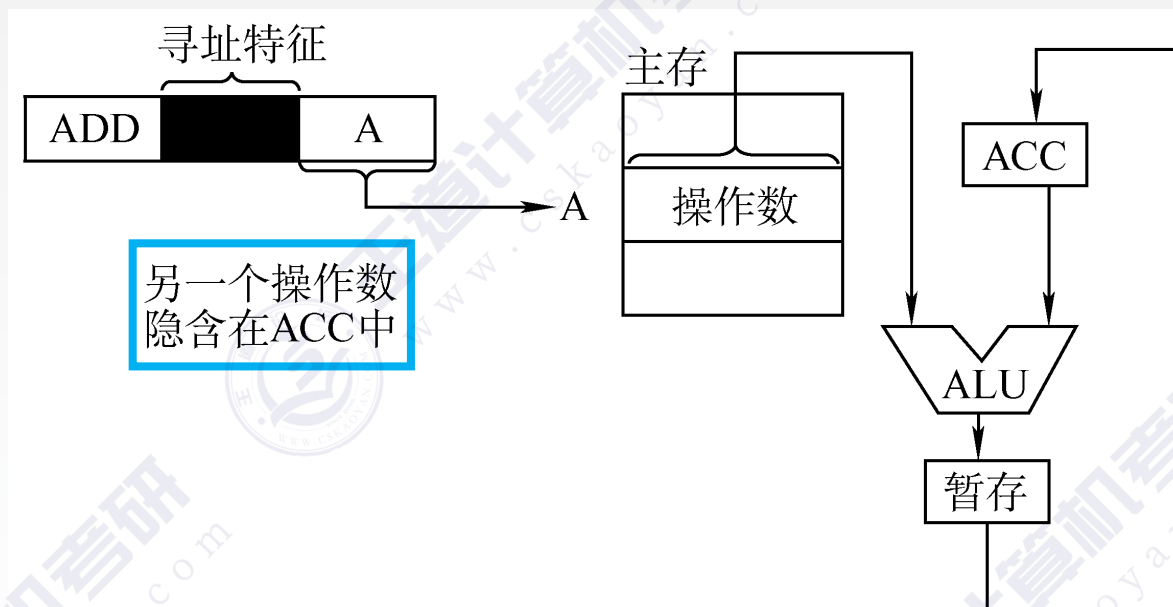
共访存2次

特点：

与一般间接寻址相比速度更快，但指令的执行阶段需要访问主存(因为操作数在主存中)。

隐含寻址

隐含寻址：不是明显地给出操作数的地址，而是在指令中隐含着操作数的地址。



优点：有利于缩短指令字长。

缺点：需增加存储操作数或隐含地址的硬件。

立即寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码 (OP)

#

0...011

立即寻址：形式地址A就是操作数本身，又称为立即数，一般采用补码形式。

#表示立即寻址特征。

一条指令的执行：

取指令 访存1次

执行指令 访存0次

暂不考虑存结果

共访存1次

优点：指令执行阶段不访问主存，指令执行时间最短

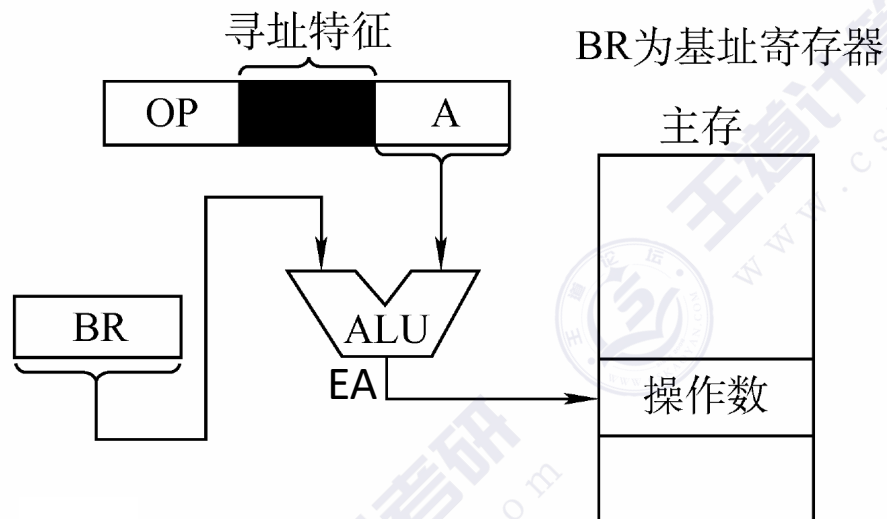
缺点：

A的位数限制了立即数的范围。

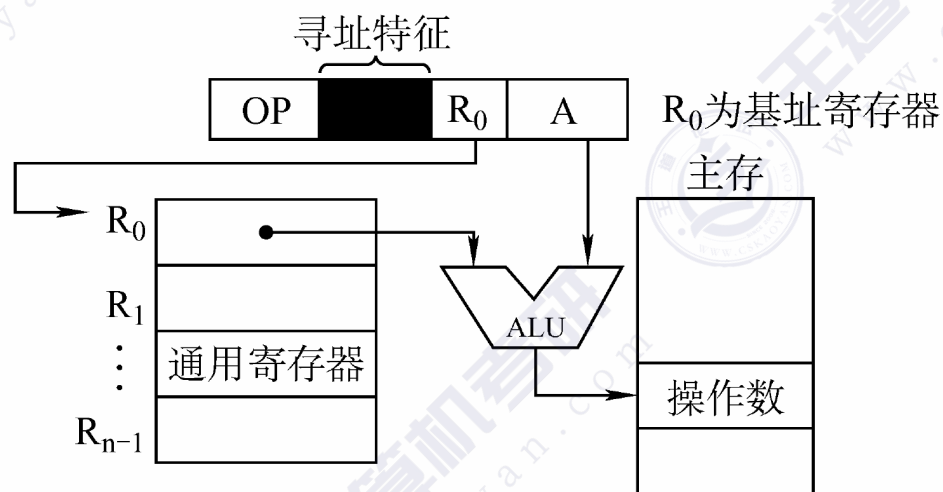
如A的位数为n，且立即数采用补码时，可表示的数据范围为 $-2^{n-1} \sim 2^{n-1} - 1$

基址寻址

基址寻址：将CPU中**基址寄存器（BR）**的内容加上指令格式中的形式地址A，而形成操作数的有效地址，即 $EA=(BR)+A$ 。



(a) 采用专用寄存器BR作为基址寄存器



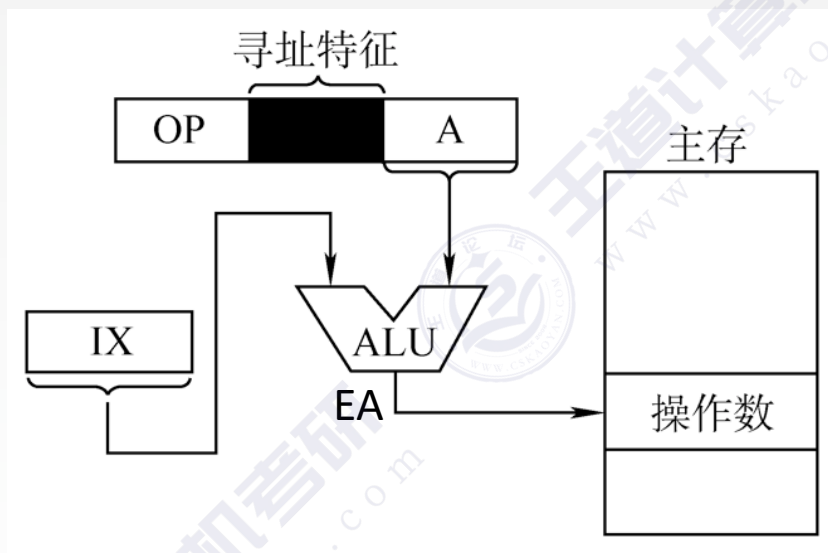
(b) 采用通用寄存器作为基址寄存器

注：基址寄存器是**面向操作系统的**，其**内容由操作系统或管理程序确定**。在程序执行过程中，基址寄存器的内容不变（作为基地址），形式地址可变（作为偏移量）。当采用通用寄存器作为基址寄存器时，可由**用户决定哪个寄存器作为基址寄存器**，但其**内容仍由操作系统确定**。

优点：可扩大寻址范围（基址寄存器的位数大于形式地址A的位数）；用户不必考虑自己的程序存于主存的哪一空间区域，故**有利于多道程序设计**，以及可用于**编制浮动程序**（整个程序在内存里边的浮动）。

变址寻址

变址寻址：有效地址EA等于指令字中的形式地址A与变址寄存器IX的内容相加之和，即 $EA = (IX) + A$ ，其中IX可为变址寄存器（专用），也可用通用寄存器作为变址寄存器。



注：变址寄存器是面向用户的，在程序执行过程中，变址寄存器的内容可由用户改变（作为偏移量），形式地址A不变（作为基地址）。

优点：在数组处理过程中，可设定A为数组的首地址，不断改变变址寄存器IX的内容，便可很容易形成数组中任一数据的地址，特别适合编制循环程序。

指令的寻址方式

2018_18. 按字节编址的计算机中，某double型数组A的首地址为2000H，使用变址寻址和循环结构访问数组A，保存数组下标的变址寄存器初值为0，每次循环取一个数组元素，其偏移地址为变址值乘以sizeof(double)，取完后变址寄存器内容自动加1。若某次循环所取元素的地址为2100H，则进入该次循环时变址寄存器的内容是_____。

A. 25

B. 32

C. 64

D. 100

答案： B

解析：

根据变址寻址的公式 $EA = (IX) + A$ ，则 $(IX) = 2100H - 2000H = 100H = 256$ ， $sizeof(double) = 8$ （双精度浮点数用8位字节表示），因此数组的下标为 $256 / 8 = 32$ ，答案选B。

指令的寻址方式



2017_15. 下列寻址方式中，最适合按下标顺序访问一维数组元素的是_____。

A. 相对寻址

B. 寄存器寻址

C. 直接寻址

D. 变址寻址

答案：D

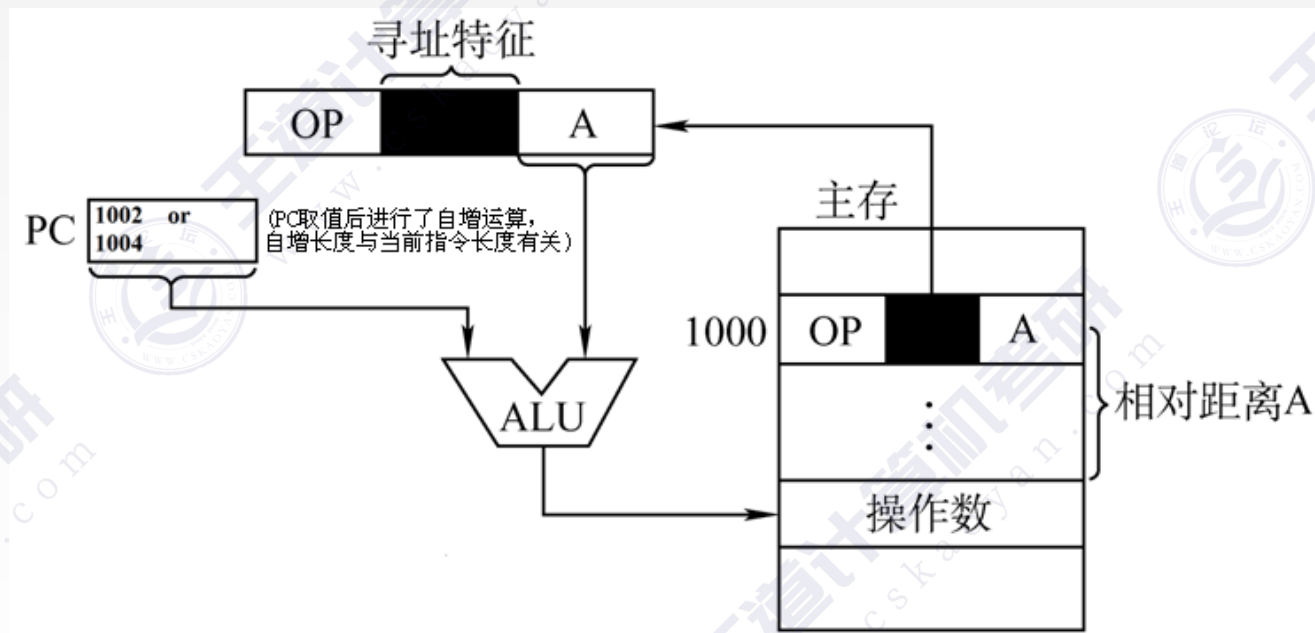
解析：

在变址操作时，将计算机指令中的地址与变址寄存器中的地址相加，得到有效地址，指令提供数组首地址，由变址寄存器来定位数据中的各元素。所以它最适合按下标顺序访问一维数组元素，选D。

相对寻址以PC为基地址，以指令中的地址为偏移量确定有效地址。寄存器寻址则是在指令中指出需要使用的寄存器。直接寻址是在指令的地址字段直接指出操作数的有效地址。

相对寻址

相对寻址：把程序计数器PC的内容加上指令格式中的形式地址A而形成操作数的有效地址，即 $EA=(PC)+A$ ，其中A是相对于PC所指地址的位移量，可正可负，补码表示。



优点：操作数的地址不是固定的，它随着PC值的变化而变化，并且与指令地址之间总是相差一个固定值，因此便于程序浮动（一段代码在程序内部的浮动）。

相对寻址广泛应用于转移指令。

指令的寻址方式

2009_16. 某机器字长为16位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一字节为操作码字段，第二字节为相对位移量字段。假定取指令时，每取一个字节PC自动加1。若某转移指令所在主存地址为2000H，相对位移量字段的内容为06H，则该转移指令成功转移后的目标地址是_____。

A. 2006H

B. 2007H

C. 2008H

D. 2009H

答案：C

解析：

相对寻址 $EA=(PC)+A$ ，首先要求的是取指令后PC的值。转移指令由两个字节组成，每取一个字节PC值自动加1，因此取指令后PC值为 $2000H+2H=2002H$ ，故 $EA=(PC)+A=2002H+06H=2008H$ 。

【易错点】本题易误选A或B。选项A没有考虑PC值的自动更新，选项B虽然考虑了PC值要自动更新，但没有注意到该转移指令是一条两字节指令，PC值仅仅“+1”而不是“+2”。

无条件转移指令：

JMP 形式地址

转移类指令通常采用相对寻址，因此执行该指令的结果是 $PC = PC + \text{形式地址}$ 。

其中，形式地址用补码表示，可正可负。而原本PC应该指向转移指令的后一条指令

各种条件转移指令

分类	指令的助忆符	检测的转移条件	功能描述
无符号数 条件转移指令	JC	CF = 1	进位位为 1, 则转移
	JNC	CF = 0	进位位为 0, 则转移
	JP/JPE	PF = 1	奇偶位为 1, 则转移
	JNP/JPO	PF = 0	奇偶位为 0, 则转移
	JE/JZ	ZF = 1	为零/相等, 则转移
	JNE/JNZ	ZF = 0	不为零/不相等, 则转移
	JA/JNBE	CF = 0 and ZF = 0	不低于等于/高于, 则转移
	JAE/JNB	CF = 0	不低于/高于等于, 则转移
	JB/JNAE	CF = 1	低于/不高于等于, 则转移
	JBE/JNA	CF = 1 or AF = 1	低于等于/不高于, 则转移

eg: 有无符号整数 a, b
if (a == b){
 指令1, 2, 3...
}

CMP a, b
JE 形式地址

(背后实际做了 a-b 的运算)
(相对寻址跳转到指令1的地址)

PSW寄存器的各个标志位

➤ 条件码:

OF (Overflow Flag) 溢出标志。溢出时为1, 否则置0。

SF (Sign Flag) 符号标志。结果为负时置1, 否则置0。

ZF (Zero Flag) 零标志, 运算结果为0时ZF位置1, 否则置0。

CF (Carry Flag) 进位/借位标志, 进位/借位时置1, 否则置0。

AF (Auxiliary carry Flag) 辅助进位标志, 记录运算时第3位 (半个字节) 产生的进位置。有进位时1, 否则置0。

PF (Parity Flag) 奇偶标志。结果操作数中1的个数为偶数时置1, 否则置0。

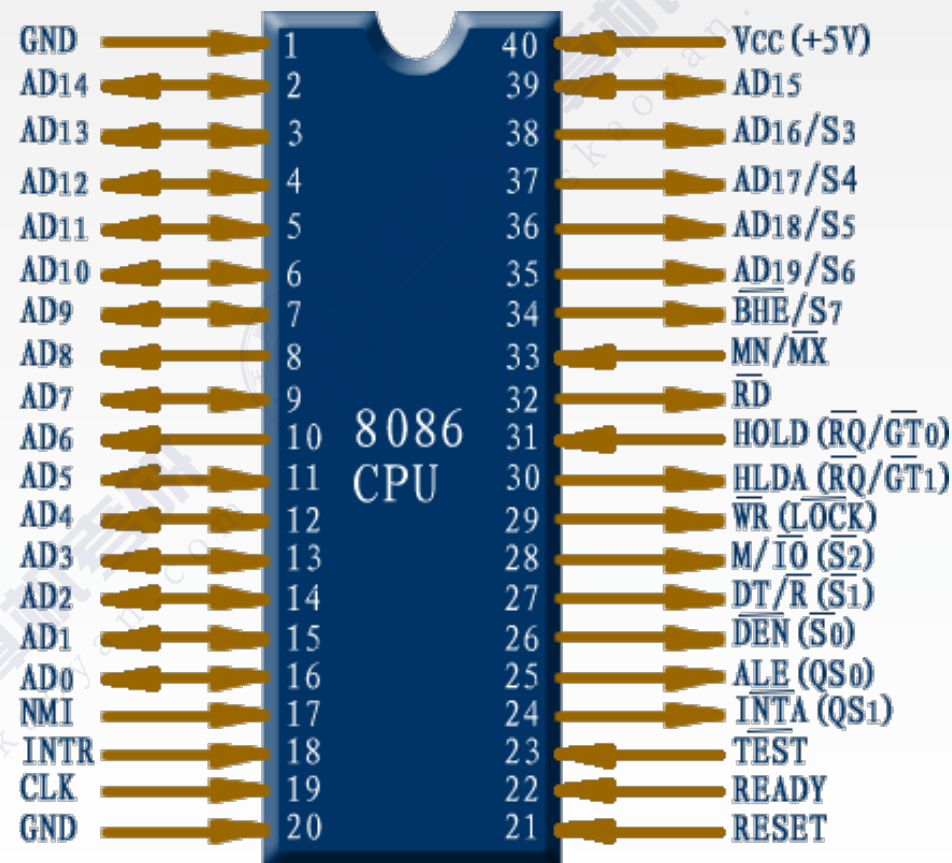
➤ 控制标志位:

DF (Direction Flag) 方向标志, 在串处理指令中控制信息的方向。

IF (Interrupt Flag) 中断标志。

TF (Trap Flag) 陷阱标志。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF



- NMI:不可屏蔽中断请求信号。常用于处理电源掉电紧急情况。
- INTR:可屏蔽中断请求信号。

各种条件转移指令

分类	指令的助忆符	检测的转移条件	功能描述
有符号数 条件转移指令	JO	OF = 1	溢出，则转移
	JNO	OF = 0	不溢出，则转移
	JS	SF = 1	为负，则转移
	JNS	SF = 0	为正，则转移
	JE/JZ	ZF = 1	为零/相等，则转移
	JNE/JNZ	ZF = 0	不为零/不相等，则转移
	JG/JNLE	ZF = 0 and SF = OF	不小于等于/大于，则转移
	JGE/JNL	SF = OF	不小于/大于等于，则转移
	JL/JNGE	SF $\neq$ OF	小于/不大于等于，则转移
	JLE/JNG	ZF = 1 or SF $\neq$ OF	小于等于/不大于，则转移

转移指令经典题目：2013年_44题

考点15

CISC 和 RISC

考点15: CISC与RISC

• 小题
17

2009



2010



2011



2012



2013



2014



2015



2016



2017



• 综合
题44

2018



2019



2020



2021



2022



2023



2024



2025



• 小题
17

历年考频: 小题×2、综合题×1

类别 对比项目	CISC	RISC
指令系统	复杂, 庞大	简单, 精简
指令数目	一般大于 200 条	一般小于 100 条
指令字长	不固定	定长
可访存指令	不加限制	只有 Load/Store 指令
各种指令执行时间	相差较大	多数指令在一个时钟周期内完成
各种指令使用频度	相差很大	都比较常用
通用寄存器数量	较少	多
目标代码	难以用优化编译生成高效的目标代码	采用优化的编译程序, 生成代码较为高效
控制方式	绝大多数为微程序控制	绝大多数为组合逻辑控制

RISC 采用指令流水线技术, 使大部分指令在一个时钟周期内完成。适合流水线的指令系统的特征有: ①指令长度应尽量一致; ②指令格式应尽量规整; ③保证除 Load/Store 指令外的其他指令都不访问存储器; ④数据和指令在存储器中“对齐”存放。

CISC与RISC

2009_17. 下列关于RISC的叙述中，错误的是_____。

- A. RISC普遍采用微程序控制器
- B. RISC大多数指令在一个时钟周期内完成
- C. RISC的内部通用寄存器数量相对CISC多
- D. RISC的指令数、寻址方式和指令格式种类相对CISC少

答案：A

解析：

相对于CISC，RISC的特点是指令条数少；指令长度固定，指令格式和寻址种类少；只有取数/存数指令访问存储器，其余指令的操作均在寄存器之间进行；CPU中通用寄存器多；大部分指令在一个或者小于一个机器周期内完成；以硬布线逻辑为主，不用或者少用微程序控制。选项B、C、D都是RISC的特点，选项A是错误的，因为RISC的速度快，所以普遍采用硬布线控制器，而非微程序控制器。

考点16

程序的机器级代码表示

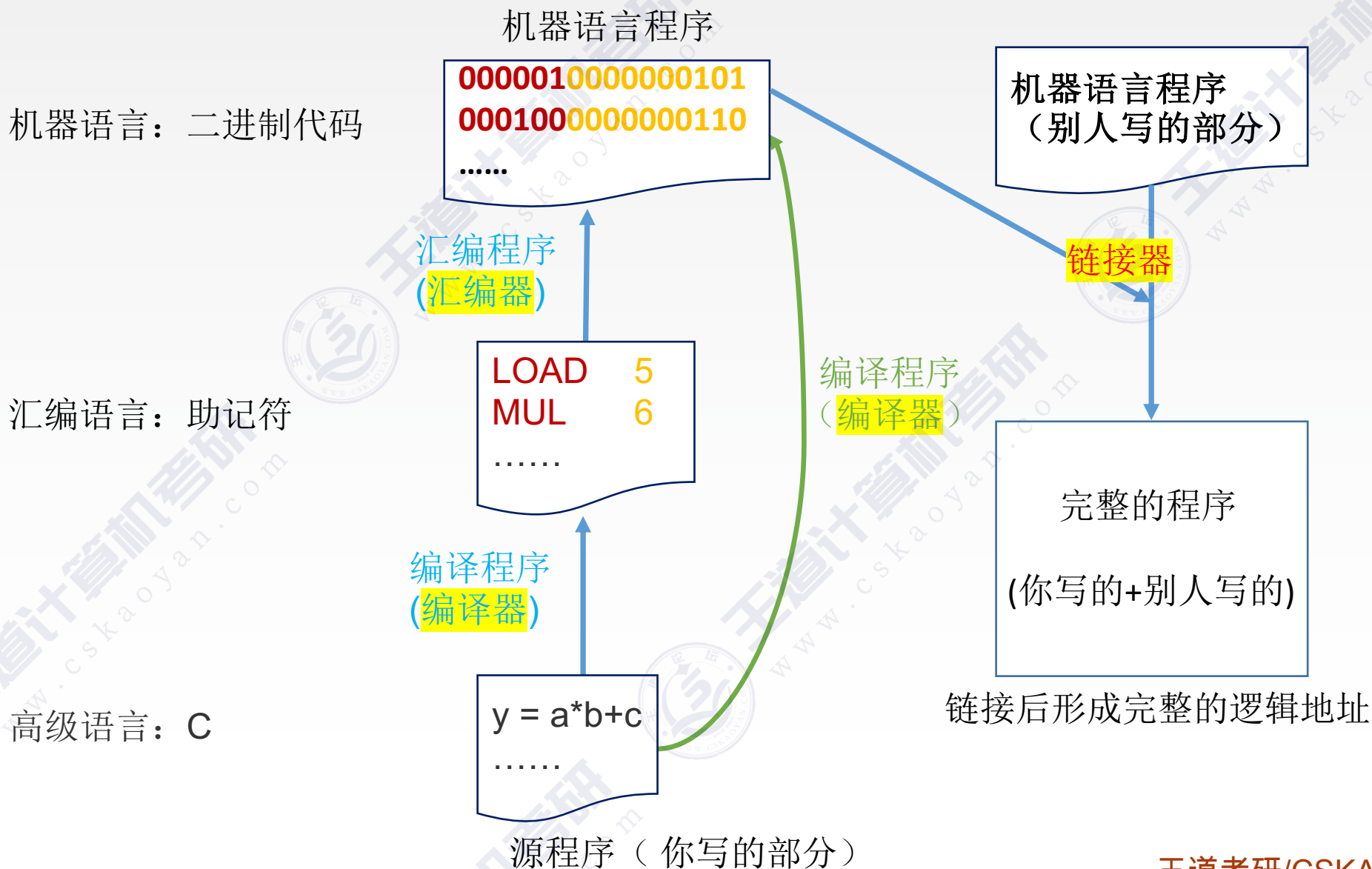
考点16：程序的机器级代码表示



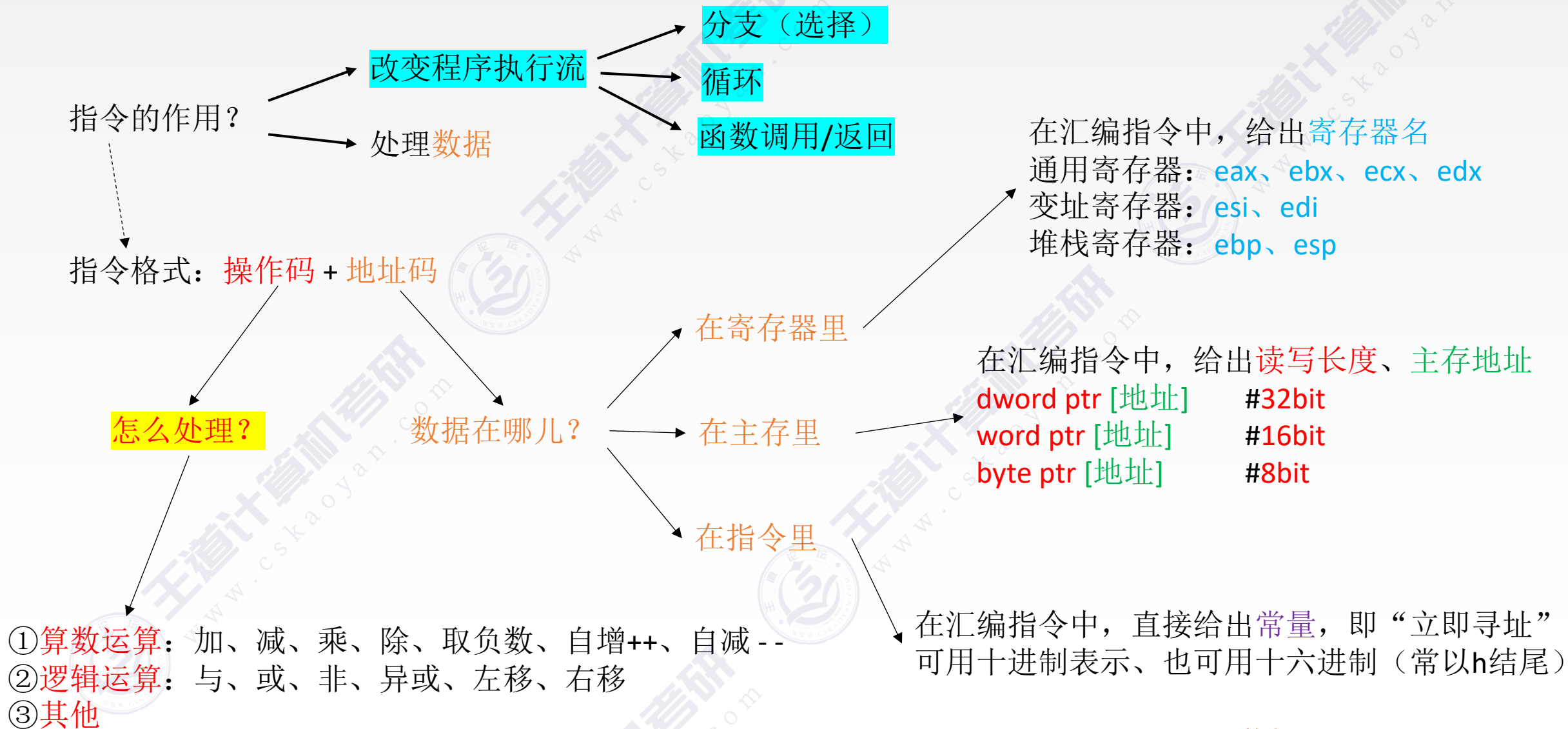
2022届新增考点

历年考频： 小题×0、 综合题×3

编译器、汇编器、链接器



汇编语言指令总结



常见的算数运算指令

destination: 目的地 (**d** 目的操作数)
source: 来源地 (**s** 源操作数)

目的操作数 **d** 不可以是常量

功能	英文	汇编指令	注释
加	add	add d,s	#计算d+s, 结果存入d
减	subtract	sub d,s	#计算d-s, 结果存入d
乘	multiply	mul d,s imul d,s	#无符号数d*s, 乘积存入d #有符号数d*s, 乘积存入d
除	divide	div s idiv s	#无符号数除法 edx:eax/s, 商存入eax, 余数存入edx #有符号数除法 edx:eax/s, 商存入eax, 余数存入edx
取负数	negative	neg d	#将d取负数, 结果存入d
自增++	increase	inc d	#将d++, 结果存入d
自减--	decrease	dec d	#将d--, 结果存入d

常见的逻辑运算指令

destination: 目的地 (**d** 目的操作数)
source: 来源地 (**s** 源操作数)

目的操作数 **d** 不可以是常量

功能	英文	汇编指令	注释
与	and	and d,s	#将 d、s 逐位相与，结果放回d
或	or	or d,s	#将 d、s 逐位相或，结果放回d
非	not	not d	#将 d 逐位取反，结果放回d
异或	exclusive or	xor d,s	#将 d、s 逐位异或，结果放回d
左移	shift left	shl d,s	#将d逻辑左移s位，结果放回d（通常s是常量）
右移	shift right	shr d,s	#将d逻辑右移s位，结果放回d（通常s是常量）

其他指令

汇编指令

注释

mov **d**, **s**

#将源操作数**s**复制到目的操作数**d**所指的位置

jmp <地址>

PC无条件转移至 <地址>

cmp a,b

jxxx <地址>

#比较a和b两个数。本质上是执行 **a-b**，并生成标志信息 OF、ZF、CF、SF
#若 cmp 指令比较的 a和b 满足条件，则PC转移至 <地址>

loop <地址>

ecx--, 若ecx!=0，则PC跳转至<地址>。通常用于实现计数型循环（循环n次）

test a,b

#将a、b**按位与**，并生成标志信息 OF、ZF、CF、SF、PF（奇偶校验位）

条件转移指令一般要和 cmp 指令一起使用

同样可以用于条件转移（简要了解即可）

循环开始前用 ecx 记录剩余循环次数

if-else 分支结构常见代码框架

`if (a > b) {
 c = a;
} else {
 c = b;
}`

“否”命题

`<=`

if 部分的逻辑

else 部分的逻辑

jxxx 指令的条件是if条件取“否”

if-else分支结构:

- ① `cmp a,b`
- ② `jxxx <⑤>`
- ③ if 部分的逻辑
- ④ `jmp <⑥>`
- ⑤ else 部分的逻辑
- ⑥ if-else完结

```
mov eax, 7  
mov ebx, 6  
cmp eax, ebx  
jle NEXT  
mov ecx, eax  
jmp END
```

NEXT:

```
mov ecx, ebx  
END:
```

#假设变量a=7, 存入eax

#假设变量b=6, 存入ebx

#比较变量a和b

#若 $a \leq b$, 转移到NEXT:

#假设用ecx存储变量c, 令 $c=a$

#无条件转移到END:

#假设用ecx存储变量c, 令 $c=b$

for/while 循环结构常见代码框架

```
int result = 0;
for(int i=1;i<=100;i++) {
    result +=i;
} //求 1+2+3+...+100
```

for/while循环结构:

- ①循环前的初始化
- ②cmp、jxxx (否命题)判断是否直接跳过循环?
- ③循环主体
- ④ cmp、jxxx (同循环条件)判断是否继续循环?

①循环前的初始化

```
mov eax,0    #用 eax 保存 result, 初值为0
mov edx,1    #用 edx 保存 i, 初始值为1
```

```
cmp edx,100  #比较 i和100
jg L2        #若 i>100, 转跳到 L2 执行
```

②是否直接跳过循环?

```
L1:          #循环主体
```

```
add eax,edx  #实现 result +=i
inc edx      #inc 自增指令, 实现 i++
```

③循环主体

```
cmp edx,100  #i和100
jle L1       #若 i<=100, 转跳到 L1 执行
```

④是否继续循环?

```
L2:          #跳出循环主体
```

补充: loop指令实现循环

使用loop 指令实现循环，必须以 ecx 作为循环计数器

```
for(int i=500;i>0;i--) {  
    做某些处理;  
} //循环500轮
```

```
mov ecx,500      #用ecx作为循环计数器  
Looptop:         #循环的开始  
...  
做某些处理  
...
```

等价于:
dec ecx
cmp ecx,0
jne Looptop

```
loop Looptop      #ecx--, 若ecx!=0, 跳转到Looptop
```

for/while循环结构:
(loop指令实现)

- ①循环前 ecx 初始化
- ②循环主体
- ③ loop <地址>

理论上，能用 loop 指令实现的功能一定
能用条件转移指令实现

其他loop指令: loopx —— 如 loopnz, loopz
loopnz——当 ecx!=0 && ZF==0 时，继续循环
loopz——当 ecx!=0 && ZF==1 时，继续循环

函数调用相关指令

函数调用指令: **call <函数名>**

- ①将**IP旧值**压栈保存（保存在函数的栈帧顶部）
- ②设置**IP新值**，无条件转移至被调用函数的第一条指令

返回指令: **ret**

从函数的栈帧顶部找到 **IP旧值**，将其出栈并恢复IP寄存器

Push 🐶 #先让esp减4，再将 🐶 压入
Pop 🐵 #栈顶元素出栈写入 🐵，再让 esp加4

注1: 🐶 可以是立即数、寄存器、主存地址

注2: 🐵 可以是寄存器、主存地址

指令: **enter**

等价

push ebp
mov ebp, esp

#保存上一层函数的栈帧基址（ebp旧值）
#设置当前函数的栈帧基址（ebp新值）

指令: **leave**

等价

mov esp, ebp
pop ebp

#让esp指向当前栈帧的底部
#将esp所指元素出栈，写入寄存器ebp

0xA00F 0034
0xA00F 0030
0xA00F 002C
0xA00F 0028
0xA00F 0024
0xA00F 0020
0xA00F 001C
0xA00F 0018
0xA00F 0014
0xA00F 0010
0xA00F 000C
0xA00F 0008
0xA00F 0004
0xA00F 0000

4字节

上一层函数栈帧基址

4字节

4字节

4字节

4字节

4字节

4字节

IP旧值(返回地址)

0xA00F 0030

← **ebp**

4字节

4字节

← **esp**

4字节

4字节

ebp: 指向当前栈帧的“**底部**”

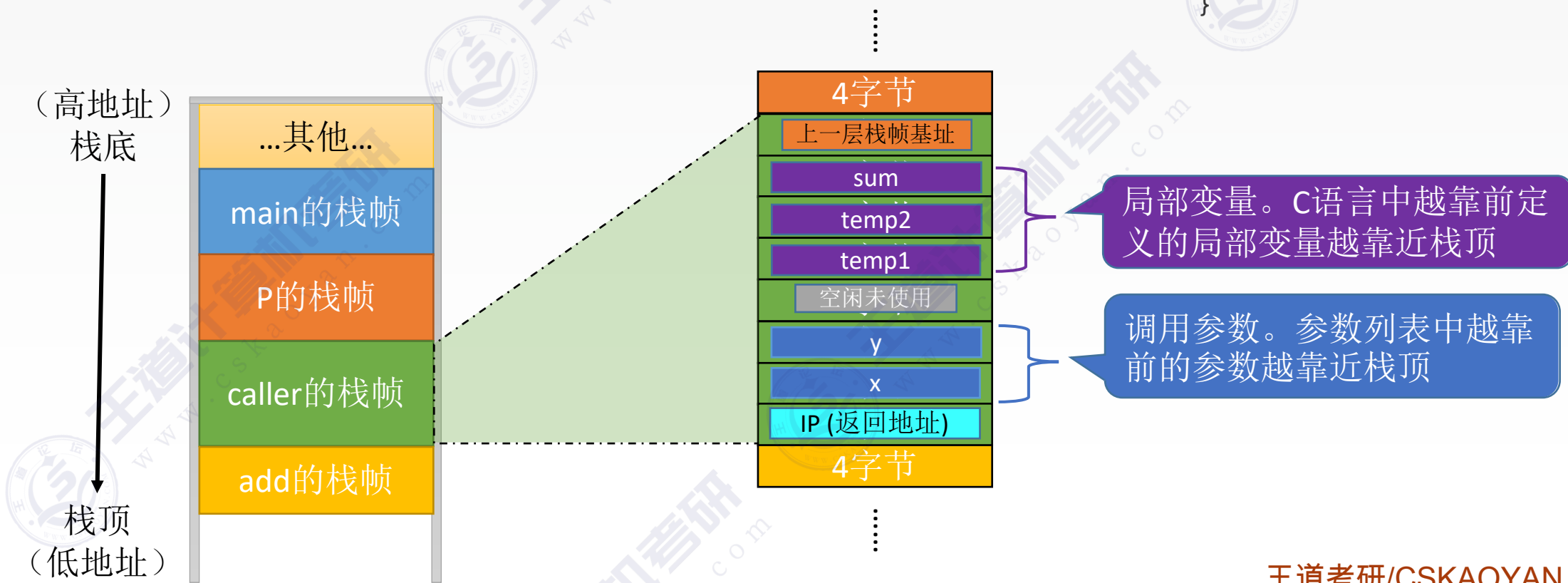
esp: 指向当前栈帧的“**顶部**”

栈帧的内部结构

- 每个栈帧大小规定为 16B 的整数倍（当前函数的栈帧除外），因此栈帧内可能出现空闲未使用的区域。
- 通常将局部变量集中存储在栈帧底部区域
- 通常将调用参数集中存储在栈帧顶部区域
- 栈帧最底部一定是上一层栈帧基址（ebp旧值）
- 栈帧最顶部一定是返回地址（当前函数的栈帧除外）

```
int caller(){
    int templ=125;
    int temp2=80;
    int sum=add(templ,temp2);
    return sum;
}

int add(int x, int y){
    return x+y;
}
```



函数调用常见代码框架

调用者:

.....

- 将调用参数写入当前栈帧的顶部区域

- 执行 **call** 指令

- 使用返回值

.....

可用 **push** 或 **mov** 指令实现

返回地址压入栈顶、并跳转到被调用函数第一条指令

通过 **eax** 寄存器

被调用者:

- 保存上一层函数栈帧, 设置当前函数栈帧

- 初始化局部变量

- 一系列处理逻辑

- 向上层函数传递返回值

- 恢复上一层函数的栈帧

- 执行 **ret** 指令

push ebp
mov ebp, esp
或: **enter** 指令

[ebp-4]、[ebp-8]...

通过 **eax** 寄存器

mov esp, ebp
pop ebp
或: **leave** 指令

从栈顶找到返回地址, 出栈并恢复 IP 值

访问局部变量: [ebp-4]、[ebp-8]...
访问参数: [ebp+8]、[ebp+12]...

push ebp
mov ebp,esp
或: enter指令

[ebp-4]、[ebp-8]...

顺序结构/分支/
循环/函数调用

通过eax寄存器

mov esp, ebp
pop ebp
或: leave指令

从栈顶找到返回地址,
出栈并恢复 IP 值

任何一个函数:

- 保存上一层函数栈帧, 设置当前函数栈帧
- 初始化局部变量
- 一系列处理逻辑
- 向上层函数传递返回值
- 恢复上一层函数的栈帧
- 执行 ret 指令

if-else分支结构:

- ① `cmp a,b`
- ② `jxxx <⑤>` (否命题)
- ③ if 部分的逻辑
- ④ `jmp <⑥>`
- ⑤ else 部分的逻辑
- ⑥ if-else 完结

for/while循环结构:

- ① 循环前的初始化
- ② `cmp、jxxx` (否命题) 判断是否直接跳过循环?
- ③ 循环主体
- ④ `cmp、jxxx` (同循环条件) 判断是否继续循环?

for/while循环结构: (loop指令实现)

- ① 循环前 ecx 初始化
- ② 循环主体
- ③ `loop <地址>`

函数调用:

- ① 将调用参数写入当前栈帧的顶部区域
- ② 执行 `call` 指令
- ③ 从 `eax` 寄存器中取得返回值

机器级代码分析步骤:

- 理解C语言代码逻辑
- 是否有局部变量?
- 是否有参数?
- 是否有分支/循环/函数调用?

2019年真题

```
int f1(int n){
    if (n>1)
        return n*f1(n-1)
    else
        return 1;
}
```

访问参数: [ebp+8]、[ebp+12]...

if-else分支结构:

- ① `cmp a,b`
- ② `jxxx <⑤>` (否命题)
- ③ if 部分的逻辑
- ④ `jmp <⑥>`
- ⑤ else 部分的逻辑
- ⑥ if-else完结

函数调用:

- ①将调用参数写入当前栈帧的顶部区域
- ②执行 `call` 指令
- ③从 `eax` 寄存器中取得返回值

45. (16分) 已知 $f(n) = n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$, 计算 $f(n)$ 的 C 语言函数 f1 的源程序 (阴影部分) 及其在 32 位计算机 M 上的部分机器级代码如下:

int	f1(int n){		
1	00401000	55	push ebp
...	...	...	...
	if(n>1)		
11	00401018	83 7D 08 01	cmp dword ptr [ebp+8],1
12	0040101C	7E 17	jle f1+35h (00401035)
	return n*f1(n-1);		
13	0040101E	8B 45 08	mov eax, dword ptr [ebp+8]
14	00401021	83 E8 01	sub eax, 1
15	00401024	50	push eax
16	00401025	E8 D6 FF FF FF	call f1 (00401000)
...	...	...	...
19	00401030	0F AF C1	imul eax, ecx
20	00401033	EB 05	jmp f1+3Ah (0040103a)
	else return 1;		
21	00401035	B8 01 00 00 00	mov eax,1
	}		
...	...	...	...
26	00401040	3B EC	cmp ebp, esp
...	...	...	...
30	0040104A	C3	ret

其中, 机器级代码行包括行号、虚拟地址、机器指令和汇编指令, 计算机 M 按字节编址, int 型数据占 32 位。请回答下列问题:

2017年真题

43. (13 分) 已知 $f(n) = \sum_{i=0}^n 2^i = 2^{n+1} - 1 = \underbrace{11 \cdots 1B}_{n+1 \text{位}}$

```
int f1(unsigned n){
    int sum=1, power=1;
    for(unsigned i=0;i<=n-1;i++){
        power *= 2;
        sum += power;
    }
    return sum;
}
```

44. (10 分) 在按字节编址的计算机 M 上, 题 43 中 f1 的部分源程序 (阴影部分) 与对应的机器级代码 (包括指令的虚拟地址) 如下:

其中, 机器级代码行包括行号、虚拟地址、机器指令和汇编指令。请回答下列问题。

```
int f1( unsigned n)
1      00401020      55      push ebp
      .....
      .....
      .....
      for( unsigned i=0; i<= n -1; i++)
      .....
      .....
20     0040105E      39 4D F4      cmp dword ptr [ ebp-0Ch ],ecx
      .....
      .....
      {
          power * = 2;
          .....
          .....
23     00401066      D1 E2      shl  edx,1
      .....
      .....
      return sum;
      .....
      .....
35     0040107F      C3      ret
```

访问局部变量: [ebp-4]、[ebp-8]...
访问参数: [ebp+8]、[ebp+12]...

for/while 循环结构:

- ① 循环前的初始化
- ② **cmp、jxxx (否命题)** 判断是否直接跳过循环?
- ③ 循环主体
- ④ **cmp、jxxx (同循环条件)** 判断是否继续循环?